

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Iulian Pavlov, FAF-241

Report

Laboratory work nr. 13

Semaphores in Your Own Kernel

Course: Operating Systems (Track B: build your own small OS (xv6))

Checked by:
Reiţman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Contents

1 Theoretical Background and Objectives **1**
1.1 Laboratory Objectives and Scope 1
1.2 Two-Step Sleep Protocol in Our xv6 Kernel 1

2 Part 1: User-Side Plumbing and Test Programs **2**
2.1 Syscall Numbers, Prototypes, Assembly Stubs, and Makefile 2
2.2 Test Programs: `user/semtest.c` and `user/mix.c` 2
2.3 Check 1: Behavior Before Kernel-Side Semaphores 5

3 Part 2: Kernel Semaphore Implementation **6**
3.1 Semaphore Table and Handlers (`kernel/sem.c`) 6
3.2 Kernel Makefile, Dispatch Table, and Git Commit 7
3.3 Check 2: Verified Ordering and Ping-Pong (`semtest`) 8

4 Part 3: `mix`, `sem_trydown`, and Analysis **8**
4.1 Running `mix` Without and With Semaphore Locking 8
4.2 Optional Extension: `sem_trydown` (Syscall 27) 9
4.3 Observe Answers (Saved in `notes.txt`) 10

5 Conclusions **11**

1 Theoretical Background and Objectives

1.1 Laboratory Objectives and Scope

Whereas Laboratories No. 11 and 12 exercised POSIX semaphores from user space on Linux, Laboratory Work No. 13 moves inside the **xv6-riscv** kernel to build counting semaphores from scratch. Each semaphore is simply an integer counter protected by a per-semaphore kernel spinlock and paired with sleep/wakeup process scheduling so blocked waiters yield the CPU cleanly.

The objectives of Laboratory Work No. 13 are:

- **User-Space Syscall Plumbing**: Assigning syscall numbers `24..26` (`SYS_sem_init`, `SYS_sem_down`, `SYS_sem_up`) alongside `SYS_trace` (23), adding prototypes in `user/user.h`, emitting RISC-V `ecall` stubs in `user/usys.pl`, adding `semtest.c` and `mix.c`, and verifying Check 1.
- **Kernel Semaphore Table (`kernel/sem.c`)**: Implementing `NSEM = 8` semaphores protected by spinlocks and the two-step calls `sleep_prepare(s)`, `release(&s->lock)`, and `sleep()`.
- **Verification and Mutual Exclusion**: Passing both ordering and ping-pong tests in `semtest`, contrasting unlocked vs. semaphore-locked output in `mix` across 3 QEMU CPUs, implementing optional `sem_trydown` (syscall 27), and committing the working tree to git.

1.2 Two-Step Sleep Protocol in Our xv6 Kernel

Unlike textbook xv6, which passes the condition lock directly to `sleep(chan, lk)`, this xv6 tree (`kernel/proc.c`) separates channel registration from parking the thread:

1. `sleep_prepare(s)` acquires `p->lock`, sets `p->chan = s`, and releases `p->lock` *while `s->lock` is still held*, so no `wakeup(s)` can slip past unseen before registration.
2. `release(&s->lock)` releases the semaphore spinlock so another hart can call `sem_up()` and `mycpu()->noff` drops from `1` back to `0`.
3. `sleep()` re-acquires `p->lock` and checks `if (p->chan != 0)`. If `wakeup(s)` arrived in the window between steps 1 and 3, `wakeup()` already cleared `p->chan = 0`, so `sleep()` returns immediately without blocking; otherwise `sleep()` sets `p->state = SLEEPING` and calls `sched()`.

Table 1 summarizes all new semaphore system calls registered in `kernel/syscall.h`.

Table 1: Semaphore System Calls Implemented in Lab 13

Constant	Number	User Prototype	Kernel Behavior
`SYS_sem_init`	24	`int sem_init(int, int);`	Initializes `sems[id].value = val` (if valid)
`SYS_sem_down`	25	`int sem_down(int);`	Decrements if `> 0`, or sleeps via `sleep_prepare/sleep`
`SYS_sem_up`	26	`int sem_up(int);`	Increments `value` and calls `wakeup(s)`
`SYS_sem_trydown`	27	`int sem_trydown(int);`	Optional: decrements if `> 0`, else returns `-1` at once

2 Part 1: User-Side Plumbing and Test Programs

2.1 Syscall Numbers, Prototypes, Assembly Stubs, and Makefile

Keeping `SYS_sync = 22` and `SYS_trace = 23` intact from Lab 10, the user-space plumbing adds:

- **`kernel/syscall.h`**: Defines `SYS_sem_init` (24), `SYS_sem_down` (25), `SYS_sem_up` (26), and optional `SYS_sem_trydown` (27).
- **`user/user.h` & `user/usys.pl`**: Declares the C wrappers and adds `entry("sem_init")`, `entry("sem_down")`, `entry("sem_up")`, and `entry("sem_trydown")` so `perl user/usys.pl` emits RISC-V `ecall` stubs into `user/usys.S`.
- **`Makefile` (`UPROGS`)**: Adds `$U/_semtest\`, `$U/_mix\`, and optional `$U/_trydown\` directly after `$U/_zombie\`.

2.2 Test Programs: `user/semtest.c` and `user/mix.c`

Listing 1: Ordering and ping-pong test suite (`user/semtest.c`)

```c
// semtest.c: two tests for the kernel semaphores (Lab 12).
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  // Test 1: order. Semaphore 0 starts at 0, so the child must wait.
  sem_init(0, 0);
  if (fork() == 0) {
    sem_down(0);
    printf("child: I ran after the parent\n");
    exit(0);
  }
```

```
  pause(10);
  printf("parent: done, I let the child go\n");
  sem_up(0);
  wait(0);

  // Test 2: ping-pong. Semaphore 1 = ping's turn, semaphore 2 = pong's turn.
  sem_init(1, 1);
  sem_init(2, 0);
  if (fork() == 0) {
    for (int i = 1; i <= 3; i++) {
      sem_down(2);
      printf(" pong %d\n", i);
      sem_up(1);
    }
    exit(0);
  }
  for (int i = 1; i <= 3; i++) {
    sem_down(1);
    printf("ping %d\n", i);
    sem_up(2);
  }
  wait(0);
  printf("semtest: OK\n");
  exit(0);
}
```

Listing 2: Concurrent console write test without and with semaphore 3 (`user/mix.c`)

```c
// mix.c: two processes print five lines each, one letter per write().
// mix      no lock: the letters of the two processes can mix
// mix lock semaphore 3 used as a lock: every line stays whole
#include "kernel/types.h"
#include "user/user.h"

void
lines(char *s, int uselock)
{
  for (int k = 0; k < 5; k++) {
    if (uselock)
      sem_down(3);
    for (char *p = s; *p; p++)
      write(1, p, 1);
    if (uselock)
      sem_up(3);
  }
}

int
main(int argc, char *argv[])
{
  int uselock = (argc > 1);
  sem_init(3, 1);
  if (fork() == 0) {
    lines("bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb\n", uselock);
    exit(0);
  }
  lines("AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\n", uselock);
  wait(0);
  exit(0);
}
```

2.3 Check 1: Behavior Before Kernel-Side Semaphores

Building and booting QEMU after Part 1 (before wiring `kernel/sem.c`) confirmed that the stubs reached kernel trap dispatch:

Listing 3: Observed Check 1 output before implementing kernel semaphores

```
$ semtest
3 semtest: unknown sys call 24
4 semtest: unknown sys call 25
child: I ran after the parent
parent: done, I let the child go
3 semtest: unknown sys call 26
3 semtest: unknown sys call 24
...
pi npongg  11
...
ppoinngg  22
...
 ppoinngg  33
...
semtest: OK
```

As expected for Check 1, unknown syscalls `24` and `25` returned `-1` immediately, so the child printed `child: I ran after the parent` *before* the parent finished `pause(10)`, and Test 2 interleaved `pi npongg 11` concurrently.

3 Part 2: Kernel Semaphore Implementation

3.1 Semaphore Table and Handlers (`kernel/sem.c`)

`kernel/sem.c` allocates a static table of `NSEM = 8 struct sem` entries shared globally across all processes. Zero-initialization in BSS starts every `spinlock` unlocked. A small shared helper `fetch_sem()` extracts argument `0` and bounds-checks `0 <= id < NSEM` across all four handlers.

Listing 4: Kernel counting semaphores with optional `sem_trydown` (`kernel/sem.c`)

```
// sem.c: counting semaphores inside the xv6 kernel (Lab 13).
// A small table of NSEM semaphores, shared by every process.
// Each one is a number plus a spinlock that protects the number.

#include "types.h"
#include "riscv.h"
#include "defs.h"
#include "param.h"
#include "spinlock.h"
#include "proc.h"

#define NSEM 8

struct sem {
  struct spinlock lock; // a zeroed spinlock is a free spinlock
  int value;            // how many may still pass
};

static struct sem sems[NSEM];

// Look up semaphore id from syscall arg 0; returns 0 if out of range.
static struct sem *
fetch_sem(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return 0;
  return &sems[id];
}

// sem_init(id, value): give semaphore id its start value.
uint64
sys_sem_init(void)
{
  struct sem *s = fetch_sem();
  int value;
  argint(1, &value);
  if (!s || value < 0)
    return -1;
  acquire(&s->lock);
  s->value = value;
  release(&s->lock);
  return 0;
}

// sem_down(id): take one unit; if there is none, sleep until there is.
uint64
sys_sem_down(void)
{
  struct sem *s = fetch_sem();
  if (!s)
    return -1;

  acquire(&s->lock);
  while (s->value == 0) {
    sleep_prepare(s);       // I will sleep on the address of s
    release(&s->lock);      // never sleep holding a spinlock
    sleep();                // returns at once if sem_up came in between
```

```
    acquire(&s->lock);
    if (killed(myproc())) { // kill must still work on a waiting process
      release(&s->lock);
      return -1;
    }
  }
  s->value--;
  release(&s->lock);
  return 0;
}

// sem_up(id): give one unit back and wake whoever sleeps on s.
uint64
sys_sem_up(void)
{
  struct sem *s = fetch_sem();
  if (!s)
    return -1;

  acquire(&s->lock);
  s->value++;
  wakeup(s);
  release(&s->lock);
  return 0;
}

// sem_trydown(id): take one unit if > 0, or return -1 immediately without sleeping.
uint64
sys_sem_trydown(void)
{
  struct sem *s = fetch_sem();
  if (!s)
    return -1;

  acquire(&s->lock);
  if (s->value == 0) {
    release(&s->lock);
    return -1;
  }
  s->value--;
  release(&s->lock);
  return 0;
}
```

3.2 Kernel Makefile, Dispatch Table, and Git Commit

To compile `kernel/sem.c` into the kernel binary and register all syscall handlers:

- **`Makefile` (`OBJS`)**: Inserted `$K/sem.o \` directly after `$K/sysproc.o \`.
- **`kernel/syscall.c`**: Declared `sys_sem_*()` prototypes, mapped `SYS_sem_init` (24) through `SYS_sem_trydown` (27) in `syscalls[]`, and added matching names in `syscallnames[]` so Lab 10's `trace` facility never reads out of bounds.
- **Git Commit (`~/xv6-riscv`)**: Committed the completed working tree to branch `riscv` at commit `aae203a`.

3.3 Check 2: Verified Ordering and Ping-Pong (`semtest`)

After rebuilding `kernel/kernel` and `fs.img` cleanly with `make`, running `semtest` inside 3-CPU QEMU produced:

Listing 5: Observed Check 2 output with kernel semaphores enabled

```
$ semtest
parent: done, I let the child go
child: I ran after the parent
ping 1
 pong 1
ping 2
 pong 2
ping 3
 pong 3
semtest: OK
```

Even though the parent sleeps for `10` ticks via `pause(10)`, the child genuinely blocks in `sys_sem_down(0)` until the parent calls `sem_up(0)`, and `ping`/`pong` alternate in lockstep across all three iterations.

4 Part 3: `mix`, `sem_trydown`, and Analysis

4.1 Running `mix` Without and With Semaphore Locking

Running `mix` twice without arguments lets parent and child interleave concurrently on 3 CPUs; passing an argument (`mix lock`) sets `uselock = 1` and wraps each 31-byte line in `sem_down(3)` / `sem_up(3)`:

Listing 6: Observed QEMU outputs of `mix` without locking (two runs)

```
$ mix
AAAAAAAAAbAbbbbAbbbAbAbAbAbAbAbAbAbAbAbAbAbbAbAbAbAbAAbAb
bAbAbA
AbAbAbAAbAbbbAAbAbAbAbbAbbAbAbAbAbAbAbAbAbAbAbAAAbAbAb
bbAbA
AbAbAbAbbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAb
bAbA
AbAAAbAbAbAbAAAAbAAbbbAbbbAbAbAbAbAbAAbAAbAbAAAbAb
bAbAbAbAbAbAbAbA
AbAbAbAbAbAbbbAbAbAbAbAbAbAbAbAbAAbAbAbAbbAbA
bbbbbbb
$ mix
AAAAAbAbAbAbAbAbAbAbAbAbAbAbAbbAbAbAbAbAbAbAbAbAbAbAbbAA
bAAAAAbAbAAAAAAbA
bbbAbbbAAbAbAbbAbAbAbAbAbAbAbAbAbAbAbAb
bAbAbAbAbAbAbAbA
AbAbAbAbAbAbAbAbAbAbAbAAbAAbbAbAbAbAbAbAbAbAb
bAbAbAbAbAbAbAbbA
AbAAbAbbbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAb
bAbAbAbAbAbAbA
AbAbAbAbAbAbAbAbAbAbAbAbAbAbAAbbbAbbAAbAbAbAbAbAbbb
bbbb
```

Listing 7: Observed QEMU output of `mix lock` (semaphore 3 initialized to 1)

```
$ mix lock
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
```

4.2 Optional Extension: `sem_trydown` (Syscall 27)

As specified in the optional exercise, `sys_sem_trydown()` (`SYS_sem_trydown = 27`) acquires `s->lock`, returns `-1` immediately when `s->value == 0` instead of calling `sleep()`, and decrements/returns `0` when a unit is available. A dedicated user program `user/trydown.c` verifies both outcomes:

Listing 8: Non-blocking semaphore test program (`user/trydown.c`)

```c
// trydown.c: demonstrate non-blocking sem_trydown() returning 0 then -1.
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  sem_init(4, 1);
  printf("first trydown: %d\n", sem_trydown(4));
  printf("second trydown (at 0): %d\n", sem_trydown(4));
  sem_up(4);
  printf("after sem_up, third trydown: %d\n", sem_trydown(4));
  exit(0);
}
```

Listing 9: Observed QEMU output of optional `trydown`

```
$ trydown
first trydown: 0
second trydown (at 0): -1
after sem_up, third trydown: 0
```

4.3 Observe Answers (Saved in `notes.txt`)

1. **Without the lock, why do the letters mix? (How many system calls does one line cost?)**
   Each call to `lines()` in `user/mix.c` prints `30` identical letters plus `'\n'` (`31` characters total) one character at a time via `for (char *p = s; *p; p++) write(1, p, 1);`. Therefore, **one line costs `31` `write()` system calls**, not a single atomic write. When `mix` runs without arguments, QEMU boots xv6 with `CPUS := 3`, allowing parent and child processes to run truly in parallel across separate CPUs (and interleave across kernel/timer transitions between individual syscalls), mixing their 1-byte console writes unpredictably.

2. **With the lock every line is whole. Which two lines of `mix.c` make one line a critical section?**
   Inside `lines()` (`user/mix.c`), `sem_down(3);` (immediately before the `for (char *p = s; *p; p++)` loop, under `if (uselock)`) and `sem_up(3);` (immediately after the loop, under `if (uselock)`) bracket all `31` character writes of one line into a single mutually exclusive critical section.

3. **In `sem_down` the test is `while (s->value == 0)`, not `if`. Why can a process wake up and still find `0`?**
   Inside `sys_sem_up()`, `wakeup(s)` scans `proc[]` (`kernel/proc.c`) and marks *every* process sleeping on channel `s` as `RUNNABLE`, not just one waiter. If multiple waiters wake together, the first woken process to re-acquire `s->lock` exits the loop and decrements `s->value` back from `1` to `0`, so any second woken process observes `s->value == 0` when it next acquires `s->lock`. Likewise, an already-running process on another hart can claim the unit first ("barging"), or `kkill()` can mark a sleeping process `RUNNABLE` without changing `s->value`. Looping with `while` guarantees `s->value` never drops below `0`.

4. **`sem_down` calls `release(&s->lock)` BEFORE `sleep()`. What would go wrong if it slept holding the spinlock?**
   Two catastrophic failures occur:

   - **Immediate kernel panic (`sched locks`):** `sleep()` acquires `p->lock` and calls `sched()` (`kernel/proc.c`), which enforces `if (mycpu()->noff != 1) panic("sched locks");`. Holding both `s->lock` and `p->lock` pushes `noff` to `2`, triggering an instant kernel panic.
   - **Deadlock / spinlock stall:** Spinlocks disable interrupts via `push_off()` (`kernel/spinlock.c`) and must be released before sleeping. Even without the `panic()` check, any caller of `sem_up()` must run `acquire(&s->lock)` before incrementing `s->value` and calling `wakeup(s)`; if the sleeping process retained `s->lock`, no `sem_up()` caller on any CPU could ever acquire `s->lock` to wake it up.

5 Conclusions

Implementing semaphores inside **xv6-riscv** connects every user-space abstraction practiced in Labs 11 and 12 down to kernel hardware and scheduler primitives:

- A semaphore reduces cleanly to three core pieces (`kernel/sem.c`): a counter `value`, a protecting spinlock `lock`, and sleeping/waking via `sleep_prepare(s)`, `sleep()`, and `wakeup(s)`.
- `semtest` verifies both one-shot parent-before-child ordering and strict multi-round ping-pong alternation (`semtest: OK`), replacing CPU-wasting busy-waiting with true blocking in `SLEEPING` state.
- `mix` demonstrates how `31` separate 1-byte `write()` calls per line race across 3 CPUs without locking, while wrapping the loop between `sem_down(3)` and `sem_up(3)` keeps every 30-character line intact.
- Re-checking `while (s->value == 0)` after waking prevents negative counts under thundering-herd wakeups, and releasing `s->lock` before `sleep()` preserves xv6's `mycpu()->noff == 1` scheduler invariant and prevents deadlock against `sem_up()`.